

SQL • DATABASE DESIGN

# Проектирование базы данных учебного центра

Модель данных, DDL, аналитические запросы и объяснение решений

PostgreSQL • нормализация • JOIN • агрегаты • индексы

**Автор портфолио: Леонид (ExpertStudy)**

31 августа 2026 • Все данные и сценарии вымышлены

**ДЕМОНСТРАЦИОННАЯ УЧЕБНАЯ РАБОТА • НЕ КЛИЕНТСКИЙ ЗАКАЗ**

## Содержание и паспорт работы

Краткая навигация по демонстрационному материалу.

**01** Предметная область и требования

**02** Логическая модель

**03** Создание схемы

**04** SQL-запросы

**05** Надёжность и выводы

### Паспорт работы

|           |                                             |
|-----------|---------------------------------------------|
| Формат    | Учебный проект небольшой реляционной БД     |
| СУБД      | PostgreSQL 15+                              |
| Объекты   | 5 таблиц, связи 1:N, ограничения и индексы  |
| Результат | Схема, DDL, примеры запросов и рекомендации |

**Статус** Материал создан как самостоятельный учебный пример для портфолио. Он показывает подход, стиль объяснения и уровень оформления; реальные заказчики и их данные не использовались.

## 01 Предметная область и требования

Учебный сценарий: учёт курсов, слушателей, записей и оплат.

Небольшому учебному центру нужна база данных, которая хранит каталог курсов, преподавателей, слушателей, факты записи и платежи. Модель должна поддерживать оперативные проверки и простую управленческую аналитику.

### Функциональные требования

- регистрировать слушателя один раз по уникальному адресу электронной почты;
- назначать преподавателя на курс и задавать вместимость группы;
- не допускать повторной записи одного слушателя на один курс;
- хранить несколько платежей по одной записи и их статусы;
- получать загрузку курсов, выручку и задолженность SQL-запросами.

### Основные бизнес-правила

| Правило                                 | Реализация в БД                              |
|-----------------------------------------|----------------------------------------------|
| Email слушателя уникален                | UNIQUE (email)                               |
| Цена неотрицательна, платёж положителен | CHECK (price >= 0), CHECK (amount > 0)       |
| Одна запись на курс                     | UNIQUE (student_id, course_id)               |
| Удаление родителя контролируется        | ON DELETE RESTRICT / CASCADE по смыслу связи |
| Статус из справочника                   | CHECK с допустимым множеством значений       |

**Допущение** Для компактности один курс связан с одним преподавателем. Расписание занятий и аудитории вынесены за границы примера.

## 02 Логическая модель

Связи построены вокруг факта записи слушателя на курс.

### Логическая модель данных

PK - первичный ключ, FK - внешний ключ

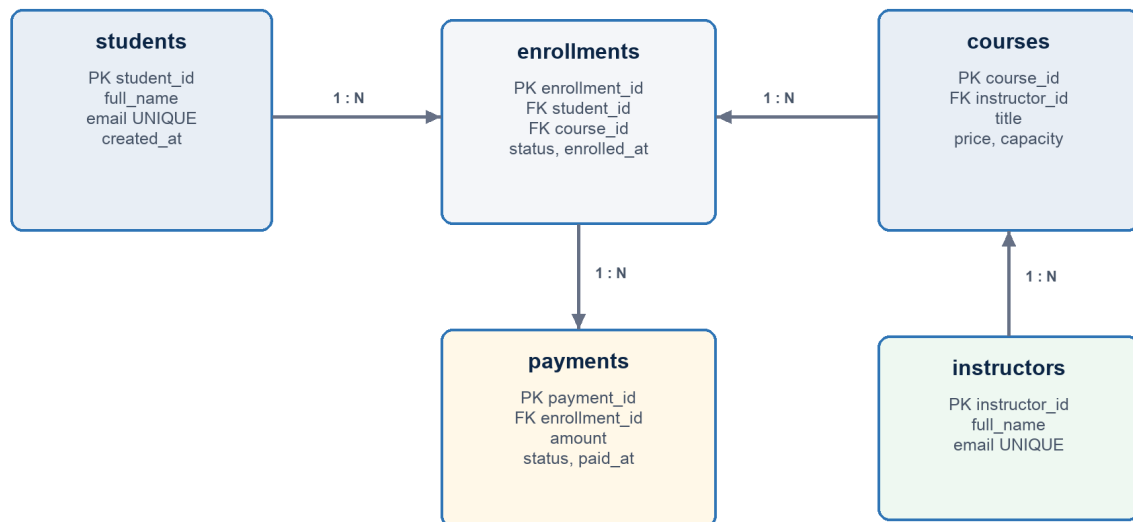


Рисунок 1. ER-диаграмма демонстрационной базы данных

### Нормализация

- 1НФ: поля атомарны, списки курсов и платежей не хранятся внутри одной ячейки;
- 2НФ: атрибуты записи зависят от enrollment\_id, а не от части составной пары;
- 3НФ: данные преподавателя вынесены из courses, платежи - из enrollments.

### Кардинальности

| Связь                    | Смысл                                               |
|--------------------------|-----------------------------------------------------|
| students 1:N enrollments | Один слушатель может записаться на несколько курсов |
| courses 1:N enrollments  | На курс записывается несколько слушателей           |
| enrollments 1:N payments | Оплату можно вносить частями                        |
| instructors 1:N courses  | Преподаватель может вести несколько курсов          |

## 03 Создание схемы

DDL включает ключи, ограничения целостности и понятные значения по умолчанию.

### Листинг 1. Справочники и каталог курсов

```
CREATE TABLE instructors (  
    instructor_id BIGSERIAL PRIMARY KEY,  
    full_name     TEXT NOT NULL,  
    email        TEXT NOT NULL UNIQUE  
);  
  
CREATE TABLE students (  
    student_id BIGSERIAL PRIMARY KEY,  
    full_name  TEXT NOT NULL,  
    email      TEXT NOT NULL UNIQUE,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE courses (  
    course_id      BIGSERIAL PRIMARY KEY,  
    instructor_id BIGINT NOT NULL REFERENCES instructors  
        ON DELETE RESTRICT,  
    title          TEXT NOT NULL,  
    price          NUMERIC(12, 2) NOT NULL CHECK (price >= 0),  
    capacity       INTEGER NOT NULL CHECK (capacity > 0)  
);
```

### Листинг 2. Записи и платежи

```
CREATE TABLE enrollments (  
    enrollment_id BIGSERIAL PRIMARY KEY,  
    student_id BIGINT NOT NULL REFERENCES students ON DELETE CASCADE,  
    course_id BIGINT NOT NULL REFERENCES courses ON DELETE RESTRICT,  
    status TEXT NOT NULL DEFAULT 'active'  
        CHECK (status IN ('active', 'completed', 'cancelled')),  
    enrolled_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
    UNIQUE (student_id, course_id)  
);  
  
CREATE TABLE payments (  
    payment_id BIGSERIAL PRIMARY KEY,  
    enrollment_id BIGINT NOT NULL REFERENCES enrollments ON DELETE CASCADE,  
    amount NUMERIC(12, 2) NOT NULL CHECK (amount > 0),  
    status TEXT NOT NULL CHECK (status IN ('pending', 'paid', 'refunded')),  
    paid_at TIMESTAMPTZ  
);
```

## 04 SQL-запросы

Примеры отвечают на типовые вопросы оператора и руководителя.

### Запрос 1. Свободные места на курсах

```
SELECT c.course_id,  
       c.title,  
       c.capacity,  
       COUNT(e.enrollment_id) FILTER (WHERE e.status = 'active') AS occupied,  
       c.capacity - COUNT(e.enrollment_id)  
         FILTER (WHERE e.status = 'active') AS seats_left  
FROM courses c  
LEFT JOIN enrollments e ON e.course_id = c.course_id  
GROUP BY c.course_id, c.title, c.capacity  
ORDER BY seats_left, c.title;
```

LEFT JOIN сохраняет в результате даже курсы без записей. FILTER учитывает только активных слушателей, а GROUP BY сворачивает строки до уровня курса.

### Запрос 2. Слушатели и оплаченная выручка

```
SELECT c.title,  
       COUNT(DISTINCT e.student_id) AS students,  
       COALESCE(SUM(p.amount) FILTER (WHERE p.status = 'paid'), 0) AS revenue  
FROM courses c  
LEFT JOIN enrollments e ON e.course_id = c.course_id  
LEFT JOIN payments p ON p.enrollment_id = e.enrollment_id  
GROUP BY c.course_id, c.title  
ORDER BY revenue DESC;
```

| Курс                      | Слушатели | Оплачено  |
|---------------------------|-----------|-----------|
| Python для анализа данных | 18        | 540 000 ₹ |
| SQL: базовый уровень      | 16        | 400 000 ₹ |
| Автоматизация с API       | 9         | 315 000 ₹ |

## 05 Расширенные запросы и производительность

В выборках учитываются частичная оплата и рост объёма таблиц.

### Запрос 3. Остаток задолженности по активным записям

```
WITH paid_by_enrollment AS (  
    SELECT enrollment_id,  
           SUM(amount) FILTER (WHERE status = 'paid') AS paid_amount  
    FROM payments  
    GROUP BY enrollment_id  
)  
SELECT s.full_name, s.email, c.title,  
       c.price - COALESCE(p.paid_amount, 0) AS debt  
FROM enrollments e  
JOIN students s ON s.student_id = e.student_id  
JOIN courses c ON c.course_id = e.course_id  
LEFT JOIN paid_by_enrollment p ON p.enrollment_id = e.enrollment_id  
WHERE e.status = 'active'  
      AND c.price > COALESCE(p.paid_amount, 0)  
ORDER BY debt DESC;
```

### Листинг 3. Индексы для частых условий

```
CREATE INDEX idx_enrollments_course_status  
ON enrollments (course_id, status);  
  
CREATE INDEX idx_payments_enrollment_paid  
ON payments (enrollment_id)  
WHERE status = 'paid';  
  
ANALYZE enrollments;  
ANALYZE payments;
```

Составной индекс поддерживает подсчёт активных записей по `course_id`. Частичный индекс `payments` меньше полного, потому что содержит только строки, участвующие в расчёте выручки.

**Проверка плана** Перед добавлением индекса нужно выполнить EXPLAIN (ANALYZE, BUFFERS) на данных, близких к реальному объёму. Индекс не является автоматическим ускорением для любой выборки.

## 06 Надёжность и выводы

База данных должна сохранять целостность не только при чтении, но и при конкурентной записи.

### Транзакционная запись

#### Листинг 4. Контур атомарной операции

```
BEGIN;  
  
INSERT INTO enrollments (student_id, course_id)  
VALUES (:student_id, :course_id)  
RETURNING enrollment_id;  
  
-- Проверка вместимости выполняется в той же транзакции.  
-- При конфликте UNIQUE или превышении capacity - ROLLBACK.  
  
COMMIT;
```

### Рекомендуемые проверки

- тест уникальности email и пары student\_id + course\_id;
- попытка записать отрицательную оплату должна завершаться ошибкой;
- удаление курса с активными записями должно блокироваться;
- сумма выручки не должна включать pending и refunded;
- права приложения следует ограничить нужными SELECT/INSERT/UPDATE.

### Итог

Схема отделяет справочники от операций, обеспечивает ссылочную целостность и поддерживает основные аналитические запросы. Для промышленной версии потребуются миграции, резервное копирование, аудит изменений и отдельная модель расписания.

**Результат примера** Показаны навыки анализа предметной области, нормализации, написания DDL/DML, работы с JOIN и агрегатами, а также базовой оптимизации PostgreSQL.